

DISPENSA DELLA LEZIONE

Data binding nei form

React

Concetto base del binding e uso controllato di input, select, radio, checkbox, textarea e form.



CAPITOLO 1

Cos'è il data binding

NOTA

Il data binding collega i dati dello stato a ciò che viene mostrato nella pagina.

Quando lo stato cambia, la UI si aggiorna in automatico.

Il data binding è il collegamento tra i dati gestiti da un componente e ciò che viene mostrato nella pagina. In React questo collegamento si costruisce di solito con lo stato: il valore vive nello stato e la UI legge quel valore per mostrarsi. Quando lo stato cambia, la parte visibile della pagina si aggiorna automaticamente.

Questo meccanismo è importante perché rende l'interfaccia prevedibile e facile da controllare. L'utente modifica un campo, React aggiorna lo stato e la pagina mostra subito il nuovo valore. Il componente diventa così il punto centrale della gestione dei dati.

CAPITOLO 2

Data binding con input

NOTA

L'input controllato legge il valore da uno stato.

onChange aggiorna lo stato a ogni digitazione.

Un input controllato è un campo il cui valore dipende dallo stato del componente. Il testo digitato dall'utente viene letto con **onChange** e salvato nello stato con **setState**. In questo modo il contenuto dell'input e quello mostrato a schermo restano sempre sincronizzati.

Questo approccio è utile quando bisogna mostrare subito il testo digitato, contare i caratteri o filtrare dati in tempo reale. L'input non viene letto solo al momento dell'invio, ma ad ogni modifica. Il componente diventa quindi la fonte unica del dato.

Esempio

```
import { useState } from "react";

function LiveText() {
  const [text, setText] = useState("");

  return (
    <div>
      <input
        type="text"
        value={text}
        onChange={(event) => setText(event.target.value)}
      />
      <p>{text}</p>
    </div>
  );
}
```

CAPITOLO 3

Data binding con select

NOTA

La select limita la scelta a opzioni definite.

Lo stato conserva il valore selezionato.

Una select controllata funziona come un input controllato, ma il valore arriva da un menu a tendina. Lo stato contiene l'opzione selezionata e `onChange` aggiorna quel valore quando l'utente sceglie una voce diversa. Il contenuto mostrato a schermo cambia subito in base alla selezione.

La select è utile quando le scelte possibili sono già definite, come una lingua, una valuta o un filtro. In questo modo l'utente non scrive un valore libero, ma seleziona una delle opzioni disponibili. Il controllo del dato resta comunque nello stato del componente.

Esempio

```
import { useState } from "react";

function PriceConverter() {
  const [currency, setCurrency] = useState("eur");
  const price = 10;

  const values = {
    eur: price,
    usd: price * 1.1,
    gbp: price * 0.85
  };

  return (
    <div>
      <select value={currency} onChange={(event) => setCurrency(event.target.value)}>
        <option value="eur">EUR</option>
        <option value="usd">USD</option>
        <option value="gbp">GBP</option>
      </select>

      <p>{values[currency].toFixed(2)}</p>
    </div>
  );
}
```

CAPITOLO 4

Data binding con radio

NOTA

I radio button permettono una sola scelta per gruppo.

Si controllano con `checked` e `onChange`.

I radio button servono quando l'utente deve scegliere una sola opzione tra più possibilità. Lo stato contiene il valore selezionato e ogni radio condivide lo stesso nome logico di scelta. Quando cambia l'opzione, cambia anche il contenuto mostrato nella pagina.

Questo tipo di controllo è adatto per scelte alternative come allineamento, dimensione del testo o tema. La logica è sempre la stessa: si memorizza il valore selezionato e si usa quel valore per decidere cosa mostrare o come formattare la UI. Ogni radio rappresenta una sola possibilità del gruppo.

Esempio

```
import { useState } from "react";

function TextSize() {
  const [size, setSize] = useState("16");

  return (
    <div>
      <label>
        <input
          type="radio"
          name="size"
          value="14"
          checked={size === "14"}
          onChange={(event) => setSize(event.target.value)}
        />
        Piccolo
      </label>

      <label>
        <input
          type="radio"
          name="size"
          value="20"
          checked={size === "20"}
          onChange={(event) => setSize(event.target.value)}
        />
        Grande
      </label>

      <p style={{ fontSize: `${size}px` }}>Testo da ridimensionare</p>
    </div>
  );
}
```

CAPITOLO 5

Data binding con checkbox

NOTA

La checkbox rappresenta uno stato vero o falso.

checked indica se è selezionata.

La checkbox serve per uno stato binario: attiva oppure disattiva. In React il valore non si legge da **value**, ma da **checked**, e la modifica passa da **onChange**. Questo rende semplice gestire conferme, opzioni visibili o stili da applicare al testo.

Le checkbox sono utili anche per abilitare o disabilitare azioni. Lo stato booleano può controllare il comportamento di un pulsante o l'aspetto di un elemento della pagina. Ogni click aggiorna un valore vero o falso nello stato.

Esempio

```
import { useState } from "react";

function TextStyleToggle() {
  const [bold, setBold] = useState(false);

  return (
    <div>
      <label>
        <input
          type="checkbox"
          checked={bold}
          onChange={(event) => setBold(event.target.checked)}
        />
        Grassetto
      </label>

      <p style={{ fontWeight: bold ? "bold" : "normal" }}>Testo target</p>
    </div>
  );
}
```

CAPITOLO 6

Data binding con textarea

NOTA

La textarea serve per testi più lunghi.

Il valore si controlla con lo stato come per l'input.

La textarea funziona come un input più adatto ai testi lunghi. Anche qui il contenuto viene controllato dallo stato e aggiornato con `onChange`. Ogni modifica rende possibile mostrare il testo scritto, il numero di caratteri o un avviso sulla lunghezza.

Con la textarea si lavora spesso su testi più estesi, come messaggi, descrizioni o commenti. Il valore viene letto dallo stato e non lasciato al solo comportamento interno del campo. In questo modo si possono mostrare feedback immediati durante la scrittura.

Esempio

```
import { useState } from "react";

function RemainingCharacters() {
  const [text, setText] = useState("");
  const maxLength = 100;

  return (
    <div>
      <textarea
        value={text}
        onChange={(event) => setText(event.target.value)}
      />
      <p>Caratteri rimanenti: {maxLength - text.length}</p>
    </div>
  );
}
```